# Project Profile — Candy Sticker Generator

**Track 1 · Development of Multimodal Content Creation Tools**

AMD AI DevMaster Hackathon 2026

---

## 1. Project Background

The anime / "chibi" sticker market is huge — stickers drive engagement on WeChat, Telegram, and Discord, and the same characters are printed on acrylic stands, badges, and T-shirts (print-on-demand). Yet producing a *consistent* 8-expression pack still requires an artist hours of manual work, and most AI tools either output inconsistent characters or depend on closed cloud APIs that cannot be deployed offline on consumer hardware.

**Candy Sticker Generator** is a local-first, multimodal tool that closes this gap. A creator types one line (or drops a reference image) and gets a complete, style-consistent sticker pack — 8 expressions, transparent, die-cut, print-ready — generated entirely on an **AMD Radeon GPU via ROCm**. No cloud API is required for any core step, so it can run air-gapped and be deployed on commodity AMD hardware.

The project deliberately targets the **AMD Radeon + ROCm** stack: it proves that a full text→image→image multimodal content-creation pipeline is practical on consumer/workstation Radeon cards, not just on NVIDIA CUDA hardware.

---

## 2. Target Users & Application Scenarios

| User | Scenario | Value |
|---|---|---|
| **Independent creators / artists** | Turn a character idea into a sellable 8-expression sticker set in minutes | 10–50× faster than manual drawing; consistent style |
| **Sticker / emoji market sellers** | Batch-produce packs for WeChat / Telegram / Discord stores | Direct ZIP export at platform specs (500×500 / 512×512) |
| **Print-on-Demand (POD) shops** | Generate transparent art for acrylic stands, badges, T-shirts | Print-ready transparent + die-cut border output |
| **Brand / IP operators** | Rapidly prototype mascot expression sets | Lock a signature style via self-trained LoRA |
| **ROCm / AMD developers** | Reference end-to-end multimodal pipeline on Radeon | Reproducible, offline, no CUDA |

The tool supports both **text-driven** creation (type a description) and **image-driven** creation (upload a reference image or a rough sketch → **Qwen2.5-VL-3B-Instruct** understands it on the same Radeon GPU → SDXL redraws the same character), forming a real **image** → **text** → **image** multimodal loop. This is the *■■■* (image-driven) path.

---

## 3. System Architecture

```
■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■
```

- Base model: `cagliostrolab/animagine-xl-4.0` (SDXL 1.0, OpenRAIL++-M), loaded in FP16.
- Sampler: Euler a (Euler Ancestral), 28 steps, CFG 5.5, 1024×1024, FP16.
- 8-expression batch: the same character prompt with 8 emotion tokens, one seed pool, ensuring intra-pack consistency ("look like one family").
- Style LoRA (optional): self-trained `stkstyle`, SDXL rank r=32 / α=16, ~178 MB after PEFT→ diffusers conversion, default weight 0.8.

### 4.2 Vision understanding — Qwen2.5-VL-3B-Instruct

- Used for the **image→text** leg of the multimodal loop.
- `image_to_char_desc(image, mode)` supports `reference` (reverse-generate a character description from a reference) and `sketch` (refine a rough drawing).
- Runs locally in FP16 (~6 GB VRAM); output is a structured character description fed back to SDXL.

### 4.3 Post-processing (offline, no rembg)

- **Background removal:** PIL + NumPy flood-fill from the 4 borders over near-white connected regions → transparent alpha (1-bit, zero halo).
- **Die-cut white border:** Euclidean Distance Transform (Felzenszwalb) over a 48 px transparent margin, expanding an equal-width (~32 px) white outline; final `GaussianBlur(radius=1)` for anti-aliasing.
- **Layout & export:** 2×4 sticker sheet (transparent), plus a ZIP at WeChat 500×500 / Telegram 512×512 / print specs.

### 4.4 Why these choices

- **No `rembg` / no cloud API** → fully offline, reproducible, deployable on AMD hardware.
- **SDXL + self-trained LoRA** → high fidelity at consumer-VRAM cost (~7 GB), style controllable.
- **Qwen2.5-VL (not vLLM)** → runs on consumer Radeon (vLLM has no official consumer-AMD support).

### 4.5 Style LoRA — self-trained on the same AMD Radeon (ROCm)

Beyond inference, the project **trains its own style LoRA on the same Radeon GPU**, proving the entire creative pipeline — not just generation — runs on AMD ROCm.

- **Base:** `cagliostrolab/animagine-xl-4.0` (SDXL). **Method:** PEFT LoRA on the UNet only (Text Encoders frozen, so prompt understanding is preserved).
- **Hyper-params:** rank `r=32` / `α=16`, 2000 steps, batch 2, lr `1e-4` (cosine), 1024², bf16, caption dropout 0.15. **Trigger word:** `stkstyle` (prepended to every caption).
- **Dataset:** 34 hand-picked anime character sheets with paired `.txt` captions (`lora/training_data/images/`).
- **Result:** a ~178 MB Diffusers adapter (`lora/models/stkstyle_lora_diffusers/`) that, when toggled in the WebUI (default weight 0.8), locks the signature "candy-neon Q-version" look.

- **Wall-clock: ~70 min (4230 s)** for 2000 steps on a single Navi 31 card — see `lora/train.log`.

The training script (`lora/train_stkstyle_lora.py`) is a manual Diffusers+PEFT SDXL loop written for ROCm (bf16, no GradScaler, frozen encoders); the full recipe and reproduction steps are in `lora/README.md`.

---

## 5. AMD Radeon GPU + ROCm Adaptation

This section is the compliance core: **every key inference step runs locally on an AMD Radeon GPU under ROCm — no NVIDIA/CUDA, no closed online API.**

| Item | Detail |
|---|---|
| GPU | AMD Radeon (Navi 31, gfx1100), 48 GB VRAM |
| Platform | ROCm 7.2 (runtime `7.2.26015`), PyTorch `2.11.0+rocm7.2` |
| Image gen | SDXL Animagine XL 4.0 — ROCm PyTorch, FP16, `to('cuda')` |
| Vision LM | Qwen2.5-VL-3B — ROCm PyTorch, FP16 |
| VRAM footprint | SDXL ~7 GB + Qwen-VL ~6 GB (both fit with large margin on 48 GB) |
| Offline | `HF_HUB_OFFLINE=1` forces pure-local inference; models cached locally |
| Reproducible | portable paths via env vars; runs on any ROCm 7.x Radeon (Navi 2x/3x) |

**ROCm-specific engineering notes**

- `low_cpu_mem_usage=True` + FP16 keeps SDXL VRAM at ~7 GB (naively it would balloon to ~47 GB

and OOM) — validated on Navi 31.

- `MIOPEN_DEBUG_DISABLE_FIND=1` skips long first-run MIOpen kernel tuning.
- `PYTORCH_HIP_ALLOC_CONF=expandable_segments:True` reduces HIP memory fragmentation.
- Xet transfer disabled (`HF_HUB_DISABLE_XET=1`) to avoid the Xet CDN 401 when pulling from the HF mirror — a ROCm / China-mirror workaround.
- **The style LoRA is trained on the same ROCm stack** (`lora/train_stkstyle_lora.py`, bf16, frozen text encoders, 2000 steps in ~70 min) — a second, independent proof that AMD Radeon handles the whole pipeline, not just inference.
- **Evidence:** `docs/evidence_gpu/` provides time-series **board-power(W) + VRAM** charts for all three ROCm workloads (SDXL generation, Qwen-VL image-to-prompt, LoRA training), plus a device snapshot identifying the hardware as **AMD Radeon / Navi 31 / 48 GB**. This proves the workload

executes on the Radeon GPU, not CPU or an N-card.

**Performance framing (honest):** measured **9.4 s/image** (SDXL, 28 steps, 1024², FP16, GPU-exclusive; three runs at 9.9 / 9.2 / 9.2 s) on Navi 31; Qwen-VL image-to-prompt **6.7 s** for a full-colour reference photo and **2.3 s** for a line sketch. ROCm raw throughput is roughly 70–80% of same-bandwidth CUDA; the value proposition is **price/performance** (AMD delivers similar VRAM at ~50–60% lower cost). No exaggerated speedup claims are made.

**Freshly-reproduced ROCm evidence (`docs/evidence_gpu/`, re-generated 2026-08-05):**

- `fig_sdxl_gpu.png` — SDXL sticker generation: board power jumps from idle **16 W** to a peak of **212 W** across the three back-to-back generations; VRAM ramps to **10,409 MB (~10.2 GB)** (SDXL FP16).
- `fig_img2prompt_gpu.png` — **Image-to-prompt (Qwen2.5-VL-3B)**, the image→text→image leg: board power rises to a peak of **159 W**; VRAM holds **8,373 MB (~8.2 GB)**.
- `fig_lora_training.png` — `stkstyle` LoRA training curve (from `lora/train.log`): loss drops from 0.13 → ~0.08, s/it converges from 34 → ~3.
- `fig_overview.png` — three ROCm workloads on the same Radeon compared in one frame.
- `device_info.txt` — hardware snapshot (`Card Model 0x744b`, `GFX gfx1100`, 48 GB VRAM, PyTorch 2.11.0+rocm7.2).
- `gpu_evidence.json` — raw time-series (machine-readable).

> Sampling note: `rocm-smi --showuse` and the sysfs `gpu_busy_percent` both report the GFX (graphics) engine activity and read 0 % for UNet compute on RDNA3 / Navi 31 — they do not cover the compute queue that HIP kernels run on. The sampler therefore records **board power draw (W)** via `rocm-smi --showpower`: idle ≈ 16 W, rising to 212 W (SDXL) and 159 W (Qwen-VL) during inference is the unambiguous proof that the Radeon GPU is doing the work (combined with the VRAM ramp that loads the model weights onto the GPU's HBM).

---

*Prepared for AMD AI DevMaster Hackathon · Track 1 · 2026-08-05*